

أنت مهندس برمجيات خبير في أنظمة التأمين الصحي، إدارة عقود مقدمي الخدمات، قوائم الأسعار، التصنيفات التأمينية، المطالبات المالية، Java، PostgreSQL، Spring Boot، JPA.

المطلوب تنفيذ تطوير شامل لكن مبسط لوحدة قوائم أسعار مقدمي الخدمات في نظام TPA، بحيث يستطيع المراجع نفسه تعديل واعتماد:

- سعر خدمة واحدة أو عدة خدمات.
- التصنيف التأميني للخدمة.
- فئة المنفعة التأمينية.
- حالة الحاجة إلى موافقة مسبقة.
- أي بيانات تعاقدية أخرى تؤثر في احتساب المطالبة.

يجب أن تكون تجربة المستخدم بسيطة جداً، بينما يتولى النظام داخلياً حفظ النسخ الزمنية، منع التداخل، حماية المطالبات التاريخية، وتسجيل جميع التغييرات.

1. الهدف الرئيسي

اجعل عملية التعديل للمراجع بهذه البساطة:

- فتح قائمة خدمات مقدم الخدمة
- تعديل السعر أو التصنيف التأميني أو كليهما
- تحديد تاريخ بدء التطبيق
- كتابة سبب التعديل
- الضغط على اعتماد التعديلات

يجب أن يتولى النظام تلقائياً:

- الاحتفاظ بالنسخة السابقة من الخدمة.
- إنشاء نسخة زمنية جديدة للخدمات المعدلة فقط.
- إنهاء النسخة السابقة في التاريخ الصحيح.
- عدم نسخ الخدمات التي لم تتغير.
- منع تداخل فترات الأسعار والتصنيفات.
- تطبيق السعر والتصنيف حسب تاريخ الخدمة.
- عدم تغيير نتائج المطالبات السابقة.
- حفظ السعر والتصنيف المستخدمين داخل المطالبة.
- تسجيل المستخدم والوقت والسبب والقيم السابقة والجديدة.
- دعم قائمة أسعار عامة وقوائم خاصة بجهات العمل.
- السماح بالتوسع مستقبلاً دون إعادة تصميم جوهر النظام.

لا تعرض للمستخدم مصطلحات تقنية مثل:

- Snapshot
- Version Record
- Effective Pricing Entity

استخدم عبارات واضحة مثل:

- السعر الحالي.
- السعر الجديد.
- التصنيف الحالي.
- التصنيف الجديد.

- تاريخ بدء التطبيق.
- السعر القادم.
- سجل تغييرات الخدمة.

2. تجربة المستخدم

2.1 شاشة قائمة الخدمات

اعرض جدولاً بسيطاً يحتوي على:

- كود الخدمة.
- اسم الخدمة.
- السعر الحالي.
- التصنيف التأميني الحالي.
- فئة المنفعة الحالية.
- هل تحتاج موافقة مسبقة؟
- السعر الجديد.
- التصنيف التأميني الجديد.
- فئة المنفعة الجديدة.
- حالة الموافقة المسبقة الجديدة.
- تاريخ التعديل القادم إن وجد.
- زر سجل التغييرات.

مثال:

الخدمة	السعر الحالي	التصنيف الحالي	السعر الجديد	التصنيف الجديد
كشف طبي	100	عيادات خارجية	110	عيادات خارجية
أشعة مقطعية	230	أشعة عادية	230	أشعة متقدمة
تحليل دم	80	مختبر	75	مختبر

يجب أن يستطيع المراجع تعديل:

- السعر فقط.
- التصنيف فقط.
- السعر والتصنيف معاً.
- عدة خدمات في عملية واحدة.

لا تنشئ نسخة جديدة لخدمة لم يتغير فيها أي حقل.

2.2 نافذة اعتماد التعديلات

عند الضغط على اعتماد التعديلات، اعرض نافذة تحتوي فقط على:

- عدد الخدمات المعدلة.
- تاريخ بدء التطبيق.
- سبب التعديل.
- نطاق التسعير.

- جهة العمل عند وجود قائمة خاصة.
- ملخص القيم التي تغيرت.
- تحذيرات التأثير المهمة.

الأزرار:

- إلغاء.
- اعتماد التعديلات.

2.3 رسالة التأكيد

اعرض رسالة واضحة:

سيتم تطبيق التعديلات ابتداءً من التاريخ المحدد.

ستبقى الأسعار والتصنيفات السابقة محفوظة للخدمات التي يكون تاريخها قبل هذا التاريخ.

لن تتغير المطالبات المعتمدة أو المساواة تلقائياً.

2.4 تاريخ واحد للعملية

تستخدم كل عملية تعديل تاريخ تطبيق واحداً لجميع الخدمات الموجودة فيها.

إذا احتاج المستخدم إلى تطبيق خدمات مختلفة في تواريخ مختلفة، ينشئ عمليات تعديل منفصلة.

3. المفهوم الصحيح للنسخة الزمنية

لا تنشئ نسخة زمنية للسعر فقط.

أنشئ نسخة زمنية كاملة للخدمة التعاقدية، لأنها قد تتضمن بيانات تؤثر في:

- التغطية.
- حصة المستفيد.
- الحد السنوي.
- الموافقة المسبقة.
- فئة المنفعة.
- الاستثناءات.
- التسعير.
- المطالبات.

استخدم مفهوماً مثل:

ProviderContractServiceVersion

بدلاً من PriceVersion فقط.

يجب أن تتضمن النسخة الزمنية على الأقل:

- id

- providerContractId •
- providerId •
- employerId nullable •
- pricingScope •
- serviceId •
- serviceCode •
- contractPrice •
- insuranceClassificationId •
- benefitCategoryId •
- requiresPreAuthorization •
- coverageReferenceId nullable •
- effectiveFrom •
- effectiveTo nullable •
- previousVersionId nullable •
- changeOperationId nullable •
- changeReason •
- createdBy •
- createdAt •
- active •
- rowVersion •

4. دعم نمطي التسعير

يجب دعم نمطين:

4.1 تسعير عام

قائمة خدمات وأسعار واحدة لمقدم الخدمة، تُطبق على جهات العمل المسموح لها باستخدام السعر العام.

استخدم:

PricingScope.GLOBAL

القواعد:

- providerId إلزامي.
- employerId فارغ.

4.2 تسعير خاص بجهة العمل

قائمة خدمات وأسعار خاصة بمقدم خدمة وجهة عمل محددة.

استخدم:

PricingScope.EMPLOYER_SPECIFIC

القواعد:

- providerId إلزامي.

• employerId إلزامي.

4.3 أولوية الاختيار

عند احتساب المطالبة:

1. تحقق من أن مقدم الخدمة مسموح لجهة عمل المستفيد في تاريخ الخدمة.
2. ابحث عن نسخة خدمة خاصة بجهة العمل.
3. إذا لم توجد، استخدم النسخة العامة فقط إذا كان fallback مسموحاً.
4. إذا لم توجد نسخة صالحة، أرجع خطأ مالياً واضحاً.

رسالة الخطأ:

لا توجد خدمة تعاقدية فعالة لهذا المقدم وجهة العمل في تاريخ الخدمة.

5. عملية تعديل موحدة

أنشئ كياناً باسم:

ProviderServiceChangeOperation

يمثل عملية تعديل واحدة يستطيع المراجع من خلالها تعديل السعر أو التصنيف أو بيانات الخدمة.

الحقول المقترحة:

id •
providerContractId •
providerId •
pricingScope •
employerId nullable •
effectiveFrom •
reason •
status •
createdBy •
createdAt •
approvedBy •
approvedAt •
appliedAt •
cancelledBy nullable •
cancelledAt nullable •
cancellationReason nullable •
retroactive •
rowVersion •

استخدم حالات بسيطة:

DRAFT •
APPROVED •
SCHEDULED •
APPLIED •

CANCELLED •

المراجع نفسه يستطيع:

- إنشاء العملية.
- تعديل الخدمات.
- اعتماد العملية.

إذا كان effectiveFrom هو تاريخ اليوم:

• تصبح العملية APPLIED بعد نجاح التنفيذ.

إذا كان effectiveFrom في المستقبل:

• تصبح العملية SCHEDULED.

لا تنفذ دورة موافقات متعددة حالياً، لكن صمم الكود بحيث يمكن إضافتها مستقبلاً.

6. تفاصيل عملية التعديل

أنشئ جدولاً باسم:

ProviderServiceChangeItem

الحقول المقترحة:

- id •
- changeOperationId •
- serviceId •
- oldServiceVersionId •
- newServiceVersionId nullable •
- oldPrice •
- newPrice •
- oldInsuranceClassificationId •
- newInsuranceClassificationId •
- oldBenefitCategoryId •
- newBenefitCategoryId •
- oldRequiresPreAuthorization •
- newRequiresPreAuthorization •
- priceVarianceAmount •
- priceVariancePercentage •
- changedFields •
- validationStatus •
- validationMessage nullable •

يجب ألا تضيف الخدمة إلى العملية إذا لم يتغير أي حقل.

يجب أن يوضح changedFields ما تم تعديله، مثل:

- PRICE •
- INSURANCE_CLASSIFICATION •

BENEFIT_CATEGORY •
PREAUTHORIZATION •
MULTIPLE_FIELDS •

7. قواعد النسخ الزمنية

عند اعتماد عملية تعديل:

1. اقرأ النسخة الفعالة الحالية للخدمة قبل تاريخ التعديل.
2. تحقق من عدم وجود نسخة مجدولة متداخلة.
3. لا تعطل القيم التاريخية للنسخة القديمة.
4. أنه النسخة السابقة عند تاريخ بدء النسخة الجديدة.
5. أنشئ نسخة جديدة بالقيم المعدلة.
6. انسخ القيم التي لم يغيرها المستخدم من النسخة السابقة.
7. اربط النسخة الجديدة بالنسخة السابقة.
8. اربط النسخة الجديدة بعملية التعديل.
9. سجل القيم القديمة والجديدة في Audit Log.

إذا كانت التواريخ من نوع LocalDate:

• يمكن أن تكون نهاية النسخة السابقة هي اليوم السابق لتاريخ النسخة الجديدة.

إذا كانت التواريخ من نوع timestamp:

استخدم فترات نصف مفتوحة:

(effectiveFrom, effectiveTo]

ويكون:

oldEffectiveTo = newEffectiveFrom

يفضل استخدام الفترات نصف المفتوحة لتجنب مشاكل نهاية اليوم.

8. اختيار النسخة الفعالة

أنشئ خدمة مستقلة باسم:

ProviderContractServiceResolver

مدخلاتها:

providerId •
employerId •
serviceId •
serviceDate •

مخرجاتها:

selectedServiceVersion •
pricingScope •
usedGlobalFallback •
resolutionReason •

يجب أن تختار نسخة تحقق:

effectiveFrom <= serviceDate

و:

effectiveTo > serviceDate

أو:

effectiveTo IS NULL

لا تستخدم تاريخ إدخال المطالبة.

لا تستخدم أحدث سجل فقط.

لا تستخدم حقل active وحده دون فحص التواريخ.

إذا وجدت أكثر من نسخة فعالة لنفس النطاق والفترة، أوقف العملية وأرجع خطأ تضارب بيانات.

9. تعديل السعر

عند تعديل السعر:

- يجب أن يكون السعر الجديد أكبر من صفر.
- استخدم BigDecimal.
- استخدم RoundingMode موحداً.
- لا تعطل السعر السابق مباشرة.
- لا تنشئ نسخة إذا كان السعر الجديد مساوياً للسعر السابق ولم تتغير بيانات أخرى.
- احسب نسبة وقيمة التغيير.

الحساب:

priceVarianceAmount = newPrice - oldPrice

إذا كان السعر الجديد أقل:

• نوع التغيير DISCOUNT

إذا كان السعر الجديد أعلى:

• نوع التغيير MARKUP

إذا كان متساوياً:

• نوع التغيير EQUAL

لا تخف الزيادة السعرية عن المراجع.

10. تعديل التصنيف التأميني

يجب أن يستطيع المراجع تعديل التصنيف التأميني من شاشة قائمة الخدمات نفسها.

عند تغيير التصنيف:

- لا تعدّل التصنيف السابق مباشرة.
- أنشئ نسخة زمنية جديدة للخدمة.
- احتفظ بالسعر السابق إذا لم يتغير.
- احتفظ بجميع الحقوق الأخرى التي لم تتغير.
- تحقق من أن التصنيف الجديد موجود وفعال.
- تحقق من توافق التصنيف مع نوع الخدمة.
- أعد تقييم فئة المنفعة والحاجة إلى الموافقة المسبقة عند الضرورة.
- اعرض تحذيراً إذا كان التغيير قد يؤثر في التغطية.

مثال:

النسخة القديمة:

- السعر: 230
- التصنيف: أشعة عادية
- الموافقة المسبقة: غير مطلوبة
- الفعالية حتى: 31 يوليو 2026

النسخة الجديدة:

- السعر: 230
- التصنيف: أشعة متقدمة
- الموافقة المسبقة: مطلوبة
- الفعالية من: 1 أغسطس 2026

مطالبة بتاريخ خدمة 25 يوليو تستخدم التصنيف القديم.

مطالبة بتاريخ خدمة 5 أغسطس تستخدم التصنيف الجديد.

11. فئة المنفعة والموافقة المسبقة

يجب ألا تعتمد الحاجة إلى الموافقة المسبقة على علاقة JPA قديمة أو محذوفة.

استخدم مصراً واضحاً ومعتمداً، مثل:

- serviceId
- insuranceClassificationId
- benefitCategoryId

requiresPreAuthorization •
coverageRuleId •

وخذ منطق تحديد الحاجة إلى الموافقة المسبقة بين:

- بوابة مقدم الخدمة.
- محرك التغطية.
- المطالبات.
- الموافقات المسبقة.
- قائمة خدمات مقدم الخدمة.

إذا تغير التصنيف التأميني، يجب أن يستطيع النظام:

- تحديث requiresPreAuthorization تلقائياً من قاعدة التصنيف، أو
- اقتراح القيمة الجديدة للمراجع، أو
- التحقق من القيمة المدخلة يدوياً.

لا تسمح بتعارض واضح بين التصنيف وقاعدة الموافقة المسبقة دون تحذير أو منع.

12. شاشة سجل تغييرات الخدمة

عند الضغط على سجل التغييرات، اعرض:

من	إلى	السعر	التصنيف	المنفعة	موافقة مسبقة	عدله	السبب
01-01-2026	31-07-2026	230	أشعة عادية	تشخيص	لا	أحمد	التصنيف السابق
01-08-2026	مستمر	230	أشعة متقدمة	تشخيص متقدم	نعم	محمد	تحديث التصنيف

اعرض أعلى الشاشة:

- النسخة الفعالة حالياً.
- النسخة المجدولة القادمة إن وجدت.
- تاريخ بدء النسخة القادمة.

لا تعرض التفاصيل التقنية إلا في شاشة تدقيق متخصصة.

13. تعديل عدة خدمات

يجب أن يستطيع المراجع اختيار عدة خدمات وتعديلها ضمن عملية واحدة.

مثال:

الخدمة	السعر الحالي	السعر الجديد	التصنيف الحالي	التصنيف الجديد
كشف	100	110	عيادات	عيادات
أشعة	230	230	عادية	متقدمة
تحليل	80	75	مختبر	مختبر

ثم يحدد:

- تاريخ التطبيق.
- السبب.

ويضغط اعتماد.

ينشئ النظام نسخة زمنية جديدة لكل خدمة تغير فيها حقل واحد على الأقل.

14. التعامل مع تعديل جدول قائم

إذا كانت هناك نسخة مستقبلية مجدولة لنفس الخدمة، لا تسمح بإنشاء نسخة متداخلة.

اعرض:

يوجد تعديل جدول لهذه الخدمة ابتداءً من التاريخ المحدد.

اسمح للمراجع بخيارين:

- تعديل العملية المجدولة الحالية إذا لم تبدأ بعد.
- إلغاء العملية المجدولة وإنشاء عملية جديدة.

لا تسمح بوجود نسختين متداخلتين لنفس:

- providerContractId
- serviceId
- pricingScope
- employerId
- effective period

15. إلغاء التعديل

قبل تاريخ التطبيق

يمكن للمراجع إلغاء العملية المجدولة.

عند الإلغاء:

- لا يتغير السعر أو التصنيف الحالي.
- ألغ النسخ المستقبلية التابعة للعملية.
- احتفظ بسجل العملية.
- سجل سبب الإلغاء.

بعد بدء التطبيق

لا تسمح بحذف النسخة أو تعديلها مباشرة.

يجب إنشاء عملية تعديل جديدة بتاريخ جديد.

16. التعديل بأثر رجعي

اجعل التعديل بأثر رجعي ممنوعاً افتراضياً.

أضف صلاحية مستقلة:

PROVIDER_SERVICE_RETROACTIVE_CHANGE

عند اختيار تاريخ سابق:

1. احسب المطالبات المتأثرة.
2. احسب الموافقات المسبقة المتأثرة.
3. اعرض العدد حسب الحالة.
4. اطلب تأكيداً إضافياً.
5. لا تعطل المطالبات المسوّاة تلقائياً.
6. لا تعطل المطالبات المعتمدة تلقائياً.
7. يمكن إعادة حساب المطالبات المسودة فقط.
8. ضع المطالبات قيد المراجعة في قائمة مراجعة.
9. لا تُلغ موافقة مسبقة صادرة تلقائياً دون إجراء صريح.

اعرض مثلاً:

- مطالبات مسودة: 5
- مطالبات قيد المراجعة: 3
- مطالبات معتمدة: 2
- مطالبات مسوّاة: 4
- موافقات مسبقة مفتوحة: 6

17. تأثير تغيير التصنيف على الموافقات السابقة

إذا أصبح التصنيف الجديد يتطلب موافقة مسبقة:

- لا تطبق الشرط الجديد على خدمة تمت قبل تاريخ بدء التعديل.
- لا ترفض مطالبة تاريخ خدمتها سابق لتاريخ التعديل.
- لا تلغ موافقات سابقة تلقائياً.
- طبق الشرط الجديد فقط على الخدمات التي يقع تاريخها في فترة النسخة الجديدة.

إذا تم إصدار موافقة مسبقة قبل التعديل لخدمة مستقبلية تقع بعد تاريخ بدء التصنيف الجديد:

- أعد تقييمها أو ضعها في قائمة مراجعة حسب سياسة العمل.
- لا تعطلها بصمت.

18. المطالبات وSnapshot المالي والتأميني

عند احتساب بند المطالبة، احفظ Snapshot كاملاً يحتوي على:

- providerContractId
- providerContractServiceVersionId
- changeOperationId

- providerId •
- employerId •
- pricingScope •
- usedGlobalFallback •
- serviceId •
- serviceCode •
- serviceDate •
- contractedUnitPrice •
- requestedUnitPrice •
- requestedAmount •
- allowedAmount •
- insuranceClassificationId •
- insuranceClassificationCode •
- benefitCategoryId •
- requiresPreAuthorization •
- preAuthorizationId nullable •
- coverageRuleId •
- coveragePercentage •
- patientSharePercentage •
- patientShare •
- payerShare •
- priceExcess •
- deductibleAmount •
- outOfPocketApplied •
- calculationVersion •
- calculatedAt •

بعد اعتماد أو تسوية المطالبة:

- لا تعتمد على السعر الحالي.
- لا تعتمد على التصنيف الحالي.
- اعرض القيم المحفوظة في Snapshot المطالبة.

أي إعادة تسعير أو إعادة تصنيف يجب أن تكون عملية صريحة ومسجلة، مع الاحتفاظ بالنتيجة السابقة.

19. قاعدة حصة المستفيد

تُثبت القاعدة التالية:

حصة المستفيد تُحسب من المبلغ المقبول، وليس من السعر المطلوب عندما يكون أعلى من السعر التعاقدى.

استخدم:

$$\text{allowedAmount} = \min(\text{requestedAmount}, \text{contractedAmount})$$

$$\text{priceExcess} = \max(\text{requestedAmount} - \text{contractedAmount}, 0)$$

$$\text{patientShare} = \text{allowedAmount} \times \text{patientSharePercentage}$$

$$\text{payerShare} = \text{allowedAmount} - \text{patientShare} - \text{deductibleAmount}$$

لا يدخل priceExcess ضمن حصة المستفيد.

مثال:

requestedAmount = 1000
contractedAmount = 800
patientSharePercentage = 20%

النتيجة:

allowedAmount = 800
patientShare = 160
payerShare = 640
priceExcess = 200

يجب ألا تكون حصة المستفيد 200.

نسبة التغطية وقواعد التحمل تُسترجع باستخدام التصنيف وفئة المنفعة الفعالة في تاريخ الخدمة، ثم تُحفظ داخل Snapshot المطالبة.

20. المطالبات السابقة

مطالبة مسودة

يمكن إعادة احتسابها عند فتحها باستخدام النسخة الصحيحة حسب تاريخ الخدمة.

يجب إظهار رسالة:

تم تحديث الحساب باستخدام بيانات الخدمة التعاقدية الفعالة في تاريخ الخدمة.

مطالبة قيد المراجعة

لا تعدّلها بصمت.

اعرض تنبيهاً أو ضعها في قائمة مراجعة إذا كانت هناك عملية رجعية تؤثر عليها.

مطالبة معتمدة

لا تُعدّل تلقائياً.

مطالبة مسوّاة أو مدفوعة

لا تُعدّل نهائياً بصورة تلقائية.

تظل مرتبطة بالSnapshot المالي والتأميني المحفوظ.

21. التحقق قبل الاعتماد

قبل اعتماد العملية، تحقق من:

- وجود مقدم الخدمة.
- وجود العقد.
- وجود الخدمة.
- وجود كود خدمة صالح.
- السعر أكبر من صفر.
- التصنيف التأميني موجود وفعال.
- فئة المنفعة موجودة وفعالة عند إلزامها.
- اتساق الموافقة المسبقة مع التصنيف.
- وجود تاريخ بدء التطبيق.
- التاريخ داخل فترة العقد.
- عدم وجود تداخل زمني.
- عدم وجود تعديل مجدول متعارض.
- وجود سبب للتعديل.
- صحة PricingScope.
- وجود employerId في السعر الخاص.
- عدم وجود employerId في السعر العام.
- عدم تكرار الخدمة داخل العملية.
- وجود تغيير فعلي واحد على الأقل.
- عدم وجود قيم مالية غير صالحة.
- عدم وجود خدمات غير مصنفة بعد الاعتماد.

أعد الأخطاء برسائل واضحة، مثل:

- السعر الجديد يجب أن يكون أكبر من صفر.
- التصنيف التأميني المحدد غير فعال.
- يجب اختيار جهة العمل للقائمة الخاصة.
- يوجد تعديل مجدول لهذه الخدمة في الفترة نفسها.
- تاريخ التطبيق خارج مدة العقد.
- لم يتم تغيير السعر أو التصنيف أو أي بيانات أخرى.
- التصنيف الجديد يتطلب موافقة مسبقة، يرجى مراجعة إعداد الخدمة.

22. الصلاحيات

استخدم صلاحيات مبسطة:

- PROVIDER_SERVICE_VIEW
- PROVIDER_SERVICE_EDIT
- PROVIDER_SERVICE_APPROVE
- PROVIDER_SERVICE_CANCEL_SCHEDULED
- PROVIDER_SERVICE_VIEW_HISTORY
- PROVIDER_SERVICE_RETROACTIVE_CHANGE
- PROVIDER_SERVICE_IMPORT
- PROVIDER_SERVICE_AUDIT_VIEW

يمكن منح المراجع حالياً:

VIEW •
EDIT •
APPROVE •
VIEW_HISTORY •

مع منع تعديل النسخ السابقة مباشرة.

23. سجل التدقيق

سجل لكل عملية ولكل خدمة:

userId •
providerId •
employerId •
pricingScope •
providerContractId •
serviceId •
serviceCode •
previousVersionId •
newVersionId •
oldPrice •
newPrice •
oldInsuranceClassificationId •
newInsuranceClassificationId •
oldBenefitCategoryId •
newBenefitCategoryId •
oldRequiresPreAuthorization •
newRequiresPreAuthorization •
effectiveFrom •
reason •
operationType •
retroactive •
createdAt •
approvedAt •
ipAddress • عند توفره

يجب أن يكون سجل التدقيق غير قابل للتعديل أو الحذف من واجهة المستخدم.

24. واجهات API

أنشئ APIs واضحة، مثل:

GET /providers/{providerId}/contract-services

تعرض الخدمات الفعالة حالياً والنسخ القادمة فقط.

POST /provider-service-changes

تنشئ عملية تعديل.

PUT /provider-service-changes/{id}

تعديل عملية DRAFT.

POST /provider-service-changes/{id}/approve

تعتمد العملية وتُنشئ النسخ الزمنية.

POST /provider-service-changes/{id}/cancel

تلغي عملية مجدولة.

GET /provider-contract-services/{serviceId}/history

تعرض سجل تغييرات الخدمة.

GET /provider-service-changes/{id}/impact

تعرض المطالبات والموافقات المتأثرة.

POST /claims/{claimId}/recalculate

تنفذ إعادة حساب صريجة ومراقبة.

لا تضع منطق اختيار السعر أو التصنيف أو الحساب المالي داخل Controller أو Mapper.

25. الخدمات البرمجية

أنشئ خدمات مستقلة:

ProviderServiceChangeService

مسؤولة عن:

- إنشاء عملية التعديل.
- تعديل المسودة.
- اعتماد العملية.
- إلغاء العملية المجدولة.
- التحقق من الصلاحيات.

ProviderContractServiceVersionService

مسؤولة عن:

- إنهاء النسخة السابقة.
- إنشاء النسخة الجديدة.

- نسخ الحقول غير المعدلة.
- منع التداخل.
- استرجاع السجل الزمني.

ProviderContractServiceResolver

مسؤولة عن:

- اختيار نسخة الخدمة الصحيحة حسب مقدم الخدمة والجهة والخدمة وتاريخ الخدمة.

InsuranceClassificationValidationService

مسؤولة عن:

- التحقق من صحة التصنيف.
- التحقق من فئة المنفعة.
- تحديد أو مراجعة الحاجة إلى الموافقة المسبقة.
- اكتشاف التعارضات.

ClaimFinancialCalculationService

مسؤولة عن:

- contractedAmount •
- allowedAmount •
- priceExcess •
- patientShare •
- payerShare •
- deductible •
- outOfPocket •

ChangeImpactService

مسؤولة عن:

- تحديد المطالبات المتأثرة.
- تحديد الموافقات المسبقة المتأثرة.
- تصنيفها حسب الحالة.

AuditService

مسؤولة عن تسجيل جميع التغييرات.

26. المعاملات والتزام

اعتماد عملية التعديل يجب أن يكون داخل Transaction واحدة.

استخدم:

- Optimistic Locking عبر rowVersion، أو
- قفلاً مناسباً للسجلات المتأثرة.

يجب منع اعتماد عمليتين متزامنتين لنفس الخدمة والفترة.

إذا حدث تعارض، أرجع رسالة واضحة:

تم تعديل هذه الخدمة بواسطة مستخدم آخر. يرجى تحديث الصفحة والمحاولة مجدداً.

27. الاستيراد من ملف

عند استيراد قائمة خدمات:

- لا تطبق التغييرات مباشرة.
- أنشئ عملية تعديل DRAFT.
- قارن القيم المستوردة بالقيم الحالية.
- أضف فقط الخدمات التي تغير فيها السعر أو التصنيف أو بيانات المنفعة.
- اعرض شاشة معاينة.

مثال:

- 20 خدمة تغير سعرها.
 - 5 خدمات تغير تصنيفها.
 - 3 خدمات تغير السعر والتصنيف.
 - خدمتان جديدتان.
 - 4 خدمات بها أخطاء.
 - 70 خدمة لم تتغير.
- لا يسمح بالاعتماد قبل معالجة الأخطاء الحرجة.

28. الترحيل

حوّل الخدمات والأسعار الحالية إلى نسخ زمنية أولية.

لكل سجل حالي:

- effectiveFrom من بداية العقد أو تاريخ موثوق.
- effectiveTo فارغ.
- previousVersionId فارغ.
- changeOperationId فارغ.
- active = true.
- الاحتفاظ بالسعر الحالي.
- الاحتفاظ بالتصنيف الحالي.
- الاحتفاظ بفئة المنفعة.
- الاحتفاظ بقيمة الموافقة المسبقة.

قبل إضافة القيود، اكتشف:

- أسعاراً صفرية أو سالبة.
- خدمات بلا تصنيف.
- تصنيفات غير فعالة.
- خدمات مكررة.
- فترات متداخلة.
- أسعار خاصة بلا جهة عمل.
- أسعار عامة مرتبطة بجهة.
- خدمات تعتمد على علاقة MedicalService قديمة.
- خدمات لا تحتوي كوداً صالحاً.

أنشئ تقرير Migration بالمشكلات قبل إصلاحها أو تجاهلها.

29. الاختبارات المطلوبة

أنشئ Unit Tests و Integration Tests تغطي على الأقل:

1. تعديل سعر خدمة واحدة.
2. تعديل تصنيف خدمة واحدة دون تغيير السعر.
3. تعديل السعر والتصنيف معاً.
4. تعديل عدة خدمات بتاريخ واحد.
5. عدم إنشاء نسخة لخدمة لم تتغير.
6. تطبيق تعديل من تاريخ اليوم.
7. جدولة تعديل مستقبلي.
8. إلغاء تعديل مجدول.
9. منع الفترات المتداخلة.
10. منع السعر الصفري أو السالب.
11. منع التصنيف غير الفعال.
12. استخدام السعر القديم قبل تاريخ التعديل.
13. استخدام السعر الجديد بعد تاريخ التعديل.
14. استخدام التصنيف القديم قبل التاريخ.
15. استخدام التصنيف الجديد بعد التاريخ.
16. تطبيق الحاجة إلى الموافقة المسبقة حسب النسخة الزمنية.
17. مطالبة مدخلة لاحقاً تستخدم بيانات تاريخ الخدمة.
18. المطالبة المساواة لا تتغير.
19. المطالبة تحفظ السعر والتصنيف المستخدمين.
20. التعديل بأثر رجعي يتطلب صلاحية.
21. حساب المطالبات المتأثرة بالتعديل الرجعي.
22. حساب الموافقات المسبقة المتأثرة.
23. السعر الخاص له الأولوية على العام.
24. استخدام السعر العام عند السماح بالfallback.
25. رفض العملية عند عدم وجود نسخة صالحة.
26. حصة المستفيد تُحسب من allowedAmount.
27. فرق السعر لا يدخل في حصة المستفيد.
28. تعديل التصنيف لا يعيد تصنيف مطالبة معتمدة تلقائياً.
29. سجل التدقيق يحتوي جميع القيم السابقة والجديدة.
30. تعديل خدمة واحدة لا ينسخ بقية الخدمات.
31. استيراد الملف ينشئ عملية DRAFT.
32. عمليتان متزامنتان لا تنشئان فترات متداخلة.

33. العثور على أكثر من نسخة فعالة يوقف التسعير.
34. تغيير التصنيف إلى تصنيف يحتاج موافقة يولد تنبيهاً صحيحاً.
35. المطالبة تستخدم فئة المنفعة الفعالة في تاريخ الخدمة.

30. قابلية التوسع

صمم الحل بحيث يدعم مستقبلاً:

- دورة موافقات متعددة.
- حدود اعتماد حسب نسبة تغيير السعر.
- موافقة خاصة عند تغيير التصنيف.
- أسعار حسب فرع مقدم الخدمة.
- أسعار حسب جهة العمل.
- أسعار حسب شبكة التأمين.
- أسعار حسب المنطقة.
- أسعار حسب العملة.
- تصنيفات متعددة المستويات.
- ربط التصنيف بالحدود السنوية.
- ربط التصنيف بالاستثناءات.
- حزم خدمات طبية.
- أسعار حسب وحدة القياس والكمية.
- تكاملات خارجية.
- إشعارات قبل بدء التعديلات المجدولة.
- تقارير أثر مالي وتأمني.

لا تنفذ هذه التوسعات الآن إلا إذا كانت مطلوبة، لكن تجنب تصميم يمنع إضافتها مستقبلاً.

31. معايير القبول

يُعتبر التنفيذ ناجحاً عندما:

- يستطيع المراجع تعديل السعر أو التصنيف من شاشة واحدة.
- يستطيع تعديل خدمة واحدة أو عدة خدمات.
- يحتاج فقط إلى تاريخ التطبيق والسبب والاعتماد.
- لا يتعامل المستخدم مع النسخ الزمنية يدوياً.
- يتم إنشاء نسخة جديدة للخدمات المعدلة فقط.
- لا تتغير الخدمات غير المعدلة.
- يُطبق السعر والتصنيف حسب تاريخ الخدمة.
- تبقى المطالبات السابقة على قيمها التاريخية.
- تُحفظ بيانات السعر والتصنيف والمنفعة داخل المطالبة.
- لا يسمح بتداخل النسخ.
- يمكن جدولة التعديلات وإلغاؤها قبل التطبيق.
- لا يسمح بتعديل نسخة مستخدمة مباشرة.
- التعديل الرجعي مراقب بصلاحية خاصة.
- السعر الخاص له الأولوية على العام.
- التصنيف التأميني يحدد قواعد التغطية والموافقة الصحيحة.
- حصة المستفيد تُحسب من المبلغ المقبول.
- فرق السعر لا يتحملها المستفيد.
- جميع التغييرات مسجلة.
- الواجهة تعرض النسخة الحالية والقادمة فقط.

- السجل الكامل متاح عند الطلب.
- جميع الاختبارات تمر بنجاح.

32. خطة التنفيذ

ابدأ بتحليل الكود الحالي قبل تعديل أى ملف.

حدد جميع المواضيع التي:

- تعدل السعر الحالي مباشرة.
- تعدل التصنيف الحالي مباشرة.
- تختار السعر دون `serviceDate`.
- تختار التصنيف دون `serviceDate`.
- تعتمد على آخر سجل أو `active` فقط.
- تستخدم `MedicalService relationship` قديمة.
- تحدد الموافقة المسبقة في أكثر من مكان.
- تحسب حصة المستفيد من `requestedAmount`.
- تضع الحساب المالي داخل `Mapper`.
- لا تحفظ `Snapshot` مالي وتأميني داخل المطالبة.
- تسمح بفترات متداخلة.

نُفذ العمل على مراحل:

1. تحليل الكيانات والخدمات الحالية.
2. تصميم جداول النسخ الزمنية.
3. كتابة `Migration`.
4. إضافة الكيانات والـ `Enums`.
5. تنفيذ `ProviderContractServiceResolver`.
6. تنفيذ عملية تعديل الخدمات.
7. تنفيذ التحقق من التصنيف التأميني.
8. تعديل منطق الموافقات المسبقة.
9. تعديل محرك المطالبات.
10. حفظ `Snapshot` المالي والتأميني.
11. إضافة سجل التدقيق.
12. تعديل الواجهة.
13. إضافة اختبارات التزامن والفترات.
14. إضافة بقية الاختبارات.
15. توثيق قواعد العمل.

استخدم `BigDecimal` لجميع القيم المالية.

استخدم `RoundingMode` ثابتاً ومركزياً.

استخدم `Transactional` عند اعتماد عملية التعديل.

لا تحذف التاريخ المالي أو التأميني.

لا تعطل النسخ السابقة بعد بدء استخدامها.

لا تكرر منطق اختيار السعر أو التصنيف أو الحساب في أكثر من خدمة.